

# D4: Analysis and Topology

Structures, filters, limits, differentiation, integration, and proof workflow

Summer School on Lean

July 16, Thursday

# Learning Goals

- recognize the structures behind Mathlib analysis: order, topology, metric, algebra;
- read basic limit statements written with `Tendsto`;
- compare epsilon-style definitions with filter-style statements;
- recognize the roles of `Continuous`, `deriv`, and `HasDerivAt`;
- understand where interval integrals and the fundamental theorem of calculus live in Mathlib;
- use a practical workflow for exploring and repairing analysis proofs.

# Four-Part Outline

|          |                                                                                                 |
|----------|-------------------------------------------------------------------------------------------------|
| Part I   | <b>Analytic Structures</b><br>real numbers, order, topology, metric spaces, filters             |
| Part II  | <b>Limits and Continuity</b><br>sequences, function limits, epsilon language vs filters         |
| Part III | <b>Differentiation</b><br>deriv, HasDerivAt, derivative rules and examples                      |
| Part IV  | <b>Integration and Workflow</b><br>interval integrals, FTC map, Mathlib search and proof repair |

# Timing

| Time        | Topic                                 |
|-------------|---------------------------------------|
| 0–10 min    | Orientation: the Mathlib analysis map |
| 10–40 min   | Part I: Analytic Structures           |
| 40–80 min   | Part II: Limits and Continuity        |
| 80–100 min  | Break                                 |
| 100–135 min | Part III: Differentiation             |
| 135–180 min | Part IV: Integration and Workflow     |

# Part I: Analytic Structures

Mathlib analysis is built from reusable structures:

- algebraic structure: operations and laws;
- order structure: inequalities, intervals, monotonicity;
- topological structure: neighborhoods, continuity, limits;
- metric structure: distance and epsilon-style statements;
- filters: a uniform language for convergence.

# The Real Line as a Structured Object

```
import Mathlib

#check Real
#check abs
#check Set.Icc

#synth Field Real
#synth LinearOrder Real
#synth TopologicalSpace Real
#synth MetricSpace Real
```

- The real line carries many structures at once.
- Many analysis theorems are stated for abstract spaces, not only `Real`.
- Type class search supplies the required structure.

Live Lean: `codeD4/D4_AnalyticStructures.lean`

# Topology and Metric Spaces

| Structure                       | Main idea                   | Typical concepts         |
|---------------------------------|-----------------------------|--------------------------|
| <code>TopologicalSpace X</code> | open sets and neighborhoods | continuity, limits       |
| <code>MetricSpace X</code>      | distance                    | epsilon-delta statements |
| <code>Filter X</code>           | eventual behavior           | convergence, limits      |

A metric space induces a topology, but many Mathlib statements are written at the topological/filter level.

# Filters as a Common Language

```
Tendsto f sourceFilter targetFilter
```

- The source filter says how inputs approach.
- The target filter says how outputs should approach.
- This one form covers sequence limits, function limits, one-sided limits, and limits at infinity.
- `Tendsto f l1 l2`: if the input follows `l1`, then the output follows `l2`.



## Part II: Limits and Continuity

The goal of this part is to compare two languages:

- epsilon language: concrete and familiar;
- filter language: abstract and uniform;
- Mathlib usually proves limits by composing `Tendsto` lemmas.

# Why Filters?

| Situation         | Traditional notation        | Filter statement                                    |
|-------------------|-----------------------------|-----------------------------------------------------|
| sequence limit    | $u_n \rightarrow L$         | <code>Tendsto u atTop (nhds L)</code>               |
| function limit    | $x \rightarrow a$           | <code>Tendsto f (nhds a) (nhds L)</code>            |
| punctured limit   | $x \rightarrow a, x \neq a$ | <code>Tendsto f (nhdsWithin a {a}ᶜ) (nhds L)</code> |
| limit at infinity | $x \rightarrow +\infty$     | <code>Tendsto f atTop (nhds L)</code>               |

# Sequence Limits: Epsilon-N vs Filters

Traditional statement:

$$u_n \rightarrow L \iff \forall \varepsilon > 0, \exists N, \forall n \geq N, |u_n - L| < \varepsilon.$$

Filter statement:

`Tendsto u atTop (nhds L)`

Here `atTop` means that the index goes to infinity.

# Function Limits: Epsilon-Delta vs Filters

Traditional statement:

$$\forall \varepsilon > 0, \exists \delta > 0, \forall x, |x - a| < \delta \Rightarrow |f(x) - L| < \varepsilon.$$

Filter statement:

$$\text{Tendsto } f \text{ (nhds } a \text{) (nhds } L \text{)}$$

The punctured version replaces `nhds a` by a punctured neighborhood filter.

# In Lean: Compose Tendsto Lemmas

```
open Filter
open scoped Topology

#check Filter.Tendsto
#check atTop
#check nhds
#check tendsto_const_nhds
#check tendsto_id
#check Tendsto.add
#check Continuous.tendsto
```

- We usually do not unfold epsilon-delta definitions.
- We combine general theorems about Tendsto.

Live Lean: `codeD4/D4_LimitsContinuity.lean`

- `Continuous f`: continuity everywhere.
- `ContinuousAt f a`: continuity at one point.
- Continuity at  $a$  means:

$$\text{Tendsto } f \text{ (nhds } a) \text{ (nhds } (f \ a))$$

Continuous functions preserve limits.

20-minute break

Differentiation in Mathlib appears in two common forms:

- `deriv f x`: the derivative as a value;
- `HasDerivAt f f' x`: a proposition saying the derivative is  $f'$  at  $x$ .



# deriv vs HasDerivAt

```
#check deriv  
#check HasDerivAt
```

- `deriv` is convenient when we want a function-like derivative.
- `HasDerivAt` is often better for proofs and rules.
- Many derivative rules are phrased as methods on `HasDerivAt`.

Live Lean: `codeD4/D4_Differentiation.lean`

```
#check HasDerivAt.add  
#check HasDerivAt.mul  
#check HasDerivAt.const_mul  
#check HasDerivAt.comp
```

- Constants and identity functions are basic examples.
- Sums and products are built by composition of derivative rules.
- Chain rule appears as composition.

# Example Targets

Suitable live examples:

- derivative of  $x \mapsto x^2$ ;
- derivative of  $x \mapsto x^3 + x$ ;
- a small chain rule example such as  $x \mapsto (x + 1)^2$ .

The goal is to read and combine Mathlib derivative rules, not to reprove differentiation from the definition.

## Part IV: Integration and Workflow

This part gives a map, not a full course in measure theory:

- interval integrals as an API;
- the fundamental theorem of calculus as a bridge between integration and differentiation;
- practical workflow for finding and repairing Mathlib proofs.

# Interval Integrals

```
#check intervalIntegral  
  
-- notation:  
-- integral x in a..b, f x
```

- Interval integrals are built on measure-theoretic integration.
- We only use the API in this lecture.
- The detailed measure theory is left as background infrastructure.

Live Lean: `codeD4/D4_Integration.lean`

# Fundamental Theorem of Calculus

Informal shape:

$$F(x) = \int_a^x f(t) dt \implies F'(x) = f(x)$$

In Mathlib this connects:

- continuity assumptions;
- interval integrals;
- derivative statements such as `HasDerivAt`.

# Proof Workflow

A practical Mathlib workflow:

- ① read the goal and local context;
- ② inspect definitions and theorem statements with `#check`;
- ③ search for API lemmas;
- ④ ask AI for a draft;
- ⑤ use Lean errors to repair theorem names and missing assumptions;
- ⑥ keep the final proof small and verified.

Live Lean: `codeD4/D4_ProofWorkflow.lean`

## Principle

AI is useful for drafts. Lean is the verifier. Mathlib search repairs the gap.

Good uses of AI:

- propose possible theorem names;
- suggest proof structure;
- explain errors in mathematical language;
- produce a first draft to be checked and corrected.



# Suggested Exercises

- Use `#check` to inspect `Tendsto`, `Continuous`, `HasDerivAt`, and `deriv`.
- Prove a constant sequence limit using `tendsto_const_nhds`.
- Prove a simple continuity statement by composing known continuous functions.
- Prove or inspect the derivative of  $x \mapsto x^2$ .
- Use Mathlib search to repair a deliberately incomplete proof.

Live Lean: `codeD4/D4_Exercises.lean`